>_ LECTURE 11

How AI Changes the Hacker's Workflow

Using LLMs as a research partner, code reader, and payload generator — and learning not to trust them blindly.

Ethical Hacking with AI — From Foundations to Exploitation

Instructor **Ibrahim Auwal**

You have the hands. Now meet the assistant.

You've built the fundamentals. This lecture is the bridge before the attacks begin.

Kali & Web Basics

Lab, HTTP, DNS

Intercepting w/ Burp

Proxy · Repeater

AI Workflow

You are here

Recon & OWASP

Lectures 12–18

LLM Security

Lectures 19–20

You have not attacked anything yet — that's deliberate. **Today is about using AI well before we do.**

What you'll be able to do

Explain what an LLM is

How it makes text, and why that's powerful yet unreliable

Drive Claude

Start a chat, attach files, hold a working session

Know what AI changes

And, just as important, what it does not

Use the three roles

Research partner · code reader · payload generator

Prompt well

Get useful output instead of confident nonsense

Catch its failures

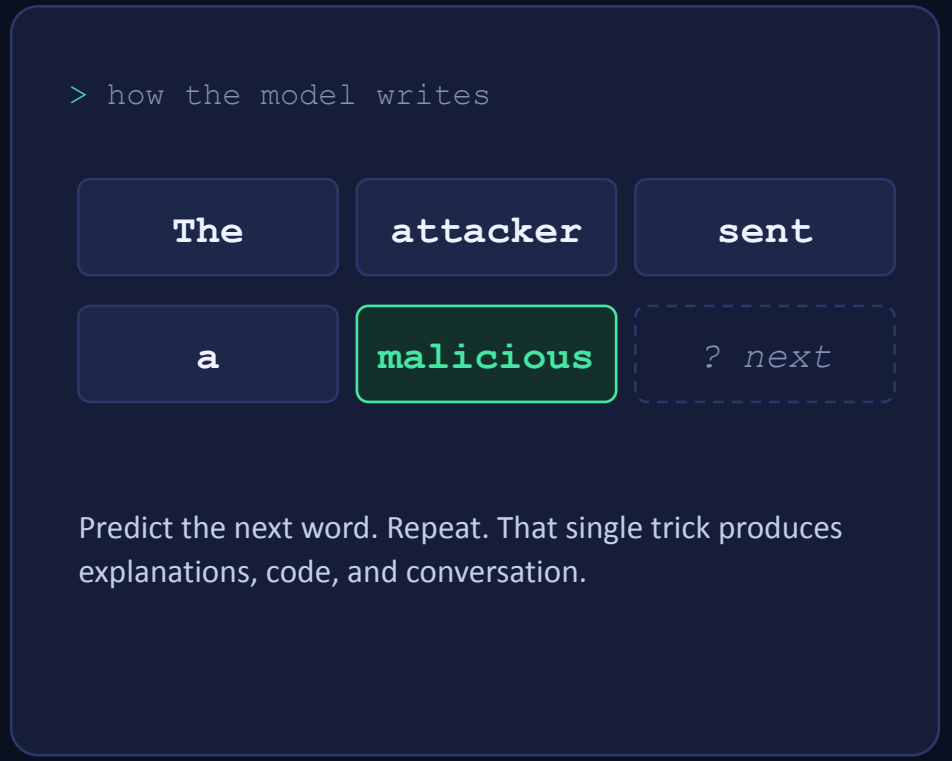
Hallucinated CVEs, fake endpoints, dead payloads

What is an LLM?

A **large language model** is a program trained to **predict text.**

It read an enormous amount of writing — books, code, docs, forums — and learned which words tend to follow which. Given some text, it predicts the next plausible chunk, then the next, one piece at a time.

ChatGPT, Claude, and Gemini are all LLMs wrapped in a chat interface.



Three consequences that matter

How it works explains everything it gets wrong. Remember these three.

01

Plausibility, not truth

It writes what sounds right. When plausible and true diverge, it states the false version just as confidently. This is where hallucination comes from.

02

A training cutoff

It knows nothing after its training date unless given web search. Ask about last month's release and it may invent an answer.

03

Not your target

It reasons about a typical system — never the exact one in front of you. The target and your proxy are the truth, not the model.

One-sentence version: a fluent, well-read pattern-matcher with no sense of truth. Treat every output as a very good guess.

Meeting your assistant

Getting in

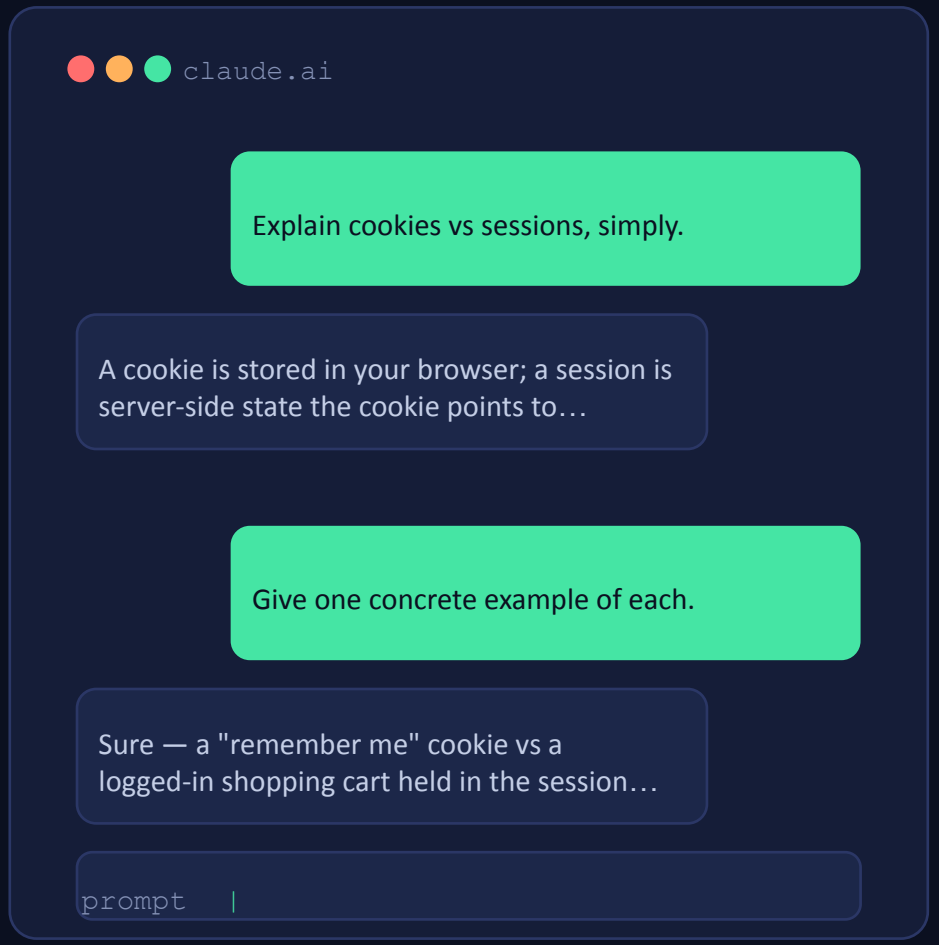
Claude runs at claude.ai and in desktop / mobile apps. Free and paid tiers exist — check support.claude.com for current specifics.

It's a conversation

You type a prompt; it replies; you keep going. There's no special syntax — a plain question is a valid prompt.

It remembers — within one chat

It can see everything above, so "make that shorter" works. A NEW chat is a blank slate. Start fresh for unrelated tasks.



Never paste real engagement data

Whatever you type into an AI tool may be transmitted to, processed by, and — depending on settings — retained by that company.

- Client source code, credentials, or internal hostnames
- Real customer data or live findings
- Anything your rules of engagement don't clearly allow

For this course: you'll only ever touch practice targets and lab data — so you're safe. But build the habit now, because it matters enormously in real work.

What AI changes — and what it doesn't

Gets faster

- **Reading unfamiliar things** code, protocols, errors, 400-line JS bundles
- **Breadth of recall** every header, SQL quirk, encoding trick
- **Boilerplate & translation** curl → Python, payload variations
- **Drafting** findings, remediation, summaries — first drafts

Doesn't change

- **Judgement on scope** the model doesn't know what you may touch
- **Verification vs the target** only the real server tells the truth
- **Understanding why** can't explain it → can't defend or adapt it
- **Accountability** "the AI told me to" is never a defence

> **An LLM is a brilliant, over-confident junior colleague who has read everything and remembers none of it precisely.**

Fast, broadly knowledgeable, occasionally brilliant — and capable of stating something completely made-up with total conviction. You'd never let that person commit to production unreviewed. Same rule here.

Three roles, ranked by risk

LOW RISK

Research partner

Explain, compare, orient

You verify against the target anyway. Point it at the right questions.

MED RISK

Code reader

First-pass review

Great at tracing data flow. Still read the code — it reasons locally.

HIGH RISK

Payload generator

Propose & mutate payloads

A hypothesis engine, never a source of truth. Test everything.

The loop that keeps you honest

The model proposes. **Burp and the target decide.** A payload it's "confident" about means nothing until the server responds.

`generate → paste → assume` is the mistake. `generate → test → observe → refine` is the discipline.

Patterns that actually work

Give context, not just a question

Stack, where input lands, what's filtered. Guessing is where hallucination lives.

Ask for reasoning first

"Reason step by step, then answer" makes wrong assumptions visible early.

Demand verification steps

"…and how would I confirm this on the target?" turns it into a lab partner.

Constrain the format

"Only the raw payload" or "as a JSON list" — less filler, less cleanup.

Make it self-critique

"Three reasons this might fail" catches its own errors.

Iterate with real data

Paste the actual 500 + stack trace back in. Concrete beats imagined.

The failure modes — all catchable

Hallucinated CVEs

A perfectly-formatted CVE that does not exist. Verify every one on the NVD.

Invented endpoints

Paths, params, functions it pattern-matched. Treat as hypotheses to test.

Dead payloads

Real filters differ from the generic one in its head. Assume a failure rate.

Confident wrong explanations

Fluent, authoritative, and backwards. Cross-check load-bearing claims.

Outdated knowledge

"As of my knowledge" hides a lot. Verify anything version-specific.

Sycophancy

Ask "is this SQLi?" and it agrees. Ask neutral questions instead.

The rule: **the model is allowed to be wrong. You are not allowed to pass its output on unverified.**

Using AI responsibly

AI changes the speed of the work, not the ethics of it. These are rules, not suggestions.

Don't leak client data

Know the tool's retention policy before you paste. Sanitise or use a no-retention tool.

AI doesn't extend your scope

If a suggested payload hits something out of scope, it stays untested. Full stop.

You own the output

Every finding carries your name. Can't explain it without the model? Not ready to ship.

Be honest about AI use

Some clients and certifications have policies. Know them and follow them.

Lab: build the habits (no attacks yet)

Everything here uses only what you have — Kali, web basics, and Burp. Nothing touches a system you don't own.

1

Calibrate trust

Ask about something you already know (cookies, status codes). How accurate is it really?

2

Find the blind spot

Have it write insecure code, then — in a fresh chat — ask it to review that code.

3

Watch the loop

Capture a normal request in Burp Repeater. Ask it to predict a change; compare to reality.

4

Catch a hallucination

Ask for CVEs on software on your Kali box. Verify each on the NVD. Document the fakes.

Deliverable ½ page: where did the model help, where did it mislead you, and what will you now always verify by hand?

What to carry forward

- An LLM predicts plausible text — it has no built-in sense of truth.
- It speeds up reading, recall, drafting, and payload variation.
- It does NOT replace judgement, verification, understanding, or accountability.
- Three roles: research partner (low) · code reader (med) · payload generator (high).
- It will hallucinate CVEs, endpoints, and working payloads — verify everything.
- Never paste real engagement data. The tool doesn't extend your authorisation.

The core loop

generate
test
observe
refine

>_ UP NEXT

Lecture 12

Reconnaissance with AI

Asset discovery, subdomain enumeration, and OSINT — using AI to parse and prioritise large, noisy tool output. You'll see immediately why "verify everything" is not optional.

Generate → Test → Observe → Refine. Bring the discipline with you.